



南開大學
Nankai University

计算机学院
并行程序设计实验报告

GPU 编程

姓名：熊诚义

学号：2313310

专业：计算机科学与技术

2025.6

目录

1 实验要求	2
2 设计思路	2
2.1 核心算法思想	2
2.2 并行算法实现策略	2
2.2.1 单 Segment 并行生成算法	2
2.2.2 多 Segment 并行拼接算法	3
2.3 算法复杂度	3
3 算法实现	3
3.1 整体架构设计	3
3.2 内存管理与数据传输	4
3.3 CUDA Kernel 实现细节	5
3.4 线程配置	6
3.5 错误处理与同步	7
4 实验结果	8
4.1 并行化效果的量化分析	8
4.2 训练阶段并行性能问题	8
5 实验总结	9
6 Github 项目链接	9

1 实验要求

1. 设计适合 Nvidia GPU 的任务分配算法，实现 CUDA 程序，在北京超算进行实验
2. 讨论一些算法/编程策略对性能的影响，如不同的线程块数、线程块大小配合不同的任务分配策略等等

2 设计思路

2.1 核心算法思想

基于 PCFG 的密码猜测算法原本是一个高度串行化的字符串拼接过程，每次需要将预计算的前缀与大量候选后缀进行组合生成完整的密码猜测。传统 CPU 实现中，这个过程需要逐一遍历所有候选值，执行字符串拷贝和拼接操作，时间复杂度为 $O(n \times m)$ ，其中 n 为候选数量， m 为平均字符串长度。

GPU 并行化的核心思想是将这个串行的字符串操作转换为大规模并行执行。每个 GPU 线程负责处理一个独立的字符串操作任务，所有线程同时执行，理论上可以将时间复杂度降低到 $O(m)$ ，实现与候选数量无关的常数时间复杂度（受 GPU 核心数限制）。

2.2 并行算法实现策略

算法流程：

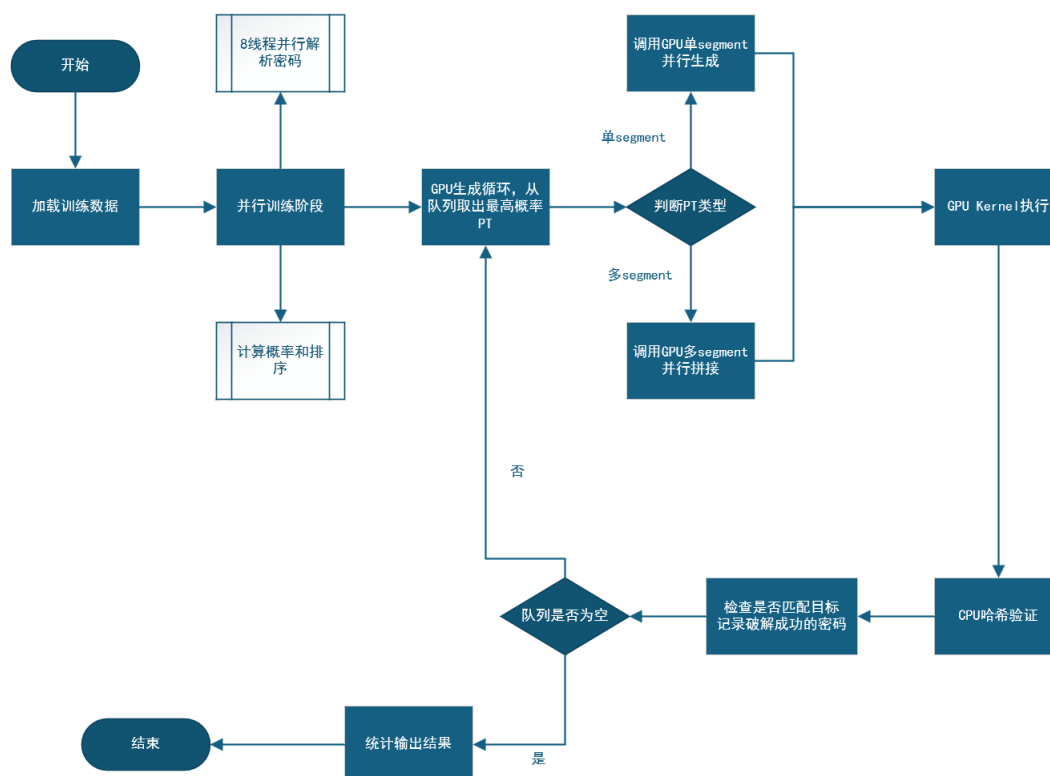


图 2.1: 算法流程图

2.2.1 单 Segment 并行生成算法

对于只包含一个 segment 的 PT，每个 GPU 线程执行以下操作：

1. 计算线程索引: $idx = blockIdx.x * blockDim.x + threadIdx.x$
2. 边界检查: 确保线程索引在有效范围内
3. 获取源字符串指针: $src = d_values[idx]$
4. 计算目标位置: $dst = d_output_buffer + d_output_offsets[idx]$
5. 执行字符逐一拷贝: 使用简单的 for 循环进行字符拷贝
6. 添加字符串终止符: $dst[len] = '\0'$

这个过程中, 每个线程完全独立工作, 无需任何同步操作。所有线程并行执行, 实现了理想的负载均衡。

2.2.2 多 Segment 并行拼接算法

对于包含多个 segment 的 PT, 需要将预计算的前缀与候选后缀进行拼接。算法设计了**两阶段拷贝策略**:

阶段一: 前缀拷贝: 每个线程首先将共同的前缀拷贝到各自的输出位置。虽然所有线程读取相同的前缀数据 (广播访问模式), 但由于前缀通常较短且会被缓存, 这种访问模式的性能影响很小。

阶段二: 后缀拼接: 每个线程读取自己对应的候选后缀, 将其拼接到前缀之后。这个阶段实现了完美的内存访问分散, 每个线程访问不同的内存位置, 最大化了内存带宽利用率。

2.3 算法复杂度

时间复杂度优化: 从串行版本的 $O(n \times m)$ 降低到并行版本的 $O(m)$, 实现了与候选数量无关的时间复杂度。在 GPU 核心数足够的情况下, 生成时间主要由字符串长度决定, 而与候选数量无关。

空间复杂度: 算法的空间复杂度为 $O(n \times m)$, 主要用于存储输入数据和输出结果。额外的空间开销 $O(n)$ 用于指针数组和偏移数组, 这个开销相对于主要数据量来说是可以接受的。

3 算法实现

3.1 整体架构设计

GPU 并行化 PCFG 算法采用了**混合架构**, 即主机端负责算法逻辑控制和数据预处理, GPU 端专门负责大规模并行的字符串生成操作。

```

1  void PriorityQueue::Generate(PT pt)
2  {
3      CalProb(pt); // 主机端概率计算
4
5      if (pt.content.size() == 1) {
6          // 单 segment 情况: 调用 GPU 并行化函数
7          gpu_generate_single_segment(a->ordered_values, temp_results);
8      } else {

```

```

9      // 多 segment 情况：主机端构建前缀，GPU 并行拼接
10     gpu_generate_multi_segment(guess, a->ordered_values, temp_results);
11 }
12 }

```

这种设计充分发挥了 CPU 和 GPU 各自的优势：CPU 擅长复杂的逻辑控制和分支处理，GPU 擅长大规模数据并行操作。

3.2 内存管理与数据传输

1. 扁平化内存布局

代码中的 `copy_strings_to_device` 函数实现了高效的字符串数据传输策略：

```

1  void copy_strings_to_device(const vector<string>& strings, char**& d_strings,
2  int*& d_lengths, char*& d_string_data) {
3      // 第一步：计算总内存需求
4      int total_chars = 0;
5      vector<int> lengths(num_strings);
6      for (int i = 0; i < num_strings; i++) {
7          lengths[i] = strings[i].length();
8          total_chars += lengths[i] + 1; // +1 for null terminator
9      }
10
11     // 第二步：分配连续的设备内存
12     CUDA_CHECK(cudaMalloc(&d_string_data, total_chars * sizeof(char)));
13     CUDA_CHECK(cudaMalloc(&d_strings, num_strings * sizeof(char*)));
14     CUDA_CHECK(cudaMalloc(&d_lengths, num_strings * sizeof(int)));
15
16     // 第三步：构建指针数组并传输数据
17     vector<char*> h_string_ptrs(num_strings);
18     int offset = 0;
19     for (int i = 0; i < num_strings; i++) {
20         h_string_ptrs[i] = d_string_data + offset;
21         CUDA_CHECK(cudaMemcpy(d_string_data + offset, strings[i].c_str(),
22                               strings[i].length() + 1, cudaMemcpyHostToDevice));
23         offset += strings[i].length() + 1;
24     }
25 }

```

这个实现的关键在于将所有字符串数据存储在一个连续的内存块 `d_string_data` 中，然后通过指针数组 `d_strings` 来索引每个字符串的起始位置。这种设计避免了 GPU 上的内存碎片，实现了最优的内存访问模式。

2. 输出缓冲区预分配

对于输出数据，算法采用了预分配策略：

```

1  // 在多 segment 函数中
2  vector<int> h_offsets(num_values);
3  int total_output_size = 0;
4  for (int i = 0; i < num_values; i++) {
5      h_offsets[i] = total_output_size;
6      total_output_size += prefix_len + values[i].length() + 1;
7  }
8
9  CUDA_CHECK(cudaMalloc(&d_output_buffer, total_output_size * sizeof(char)));
10 CUDA_CHECK(cudaMalloc(&d_output_offsets, num_values * sizeof(int)));

```

这种预分配策略的优势是避免了 GPU kernel 中的动态内存分配，所有线程都可以直接写入预先计算好的内存位置，消除了线程间的同步开销。

3.3 CUDA Kernel 实现细节

1. 单 Segment 并行生成 Kernel

```

1  __global__ void generate_single_segment_kernel(
2      char** d_values, int* d_value_lengths,
3      char* d_output_buffer, int* d_output_offsets, int num_values)
4  {
5      int idx = blockIdx.x * blockDim.x + threadIdx.x;
6
7      if (idx < num_values) {
8          char* src = d_values[idx];           // 源字符串指针
9          char* dst = d_output_buffer + d_output_offsets[idx]; // 目标位置
10         int len = d_value_lengths[idx];       // 字符串长度
11
12         // 逐字符拷贝
13         for (int i = 0; i < len; i++) {
14             dst[i] = src[i];
15         }
16         dst[len] = '\0'; // 添加字符串结束符
17     }
18 }

```

这个 kernel 的设计非常简洁高效：每个线程处理一个字符串的完整拷贝操作，所有线程完全并行执行，无需任何同步操作。线程索引计算 $\text{idx} = \text{blockIdx.x} * \text{blockDim.x} + \text{threadIdx.x}$ 确保了

线程与数据的一一对应关系。

2. 多 Segment 并行拼接 Kernel

```

1  __global__ void generate_multi_segment_kernel(
2      char* d_prefix, int prefix_length,
3      char** d_values, int* d_value_lengths,
4      char* d_output_buffer, int* d_output_offsets, int num_values)
5  {
6      int idx = blockIdx.x * blockDim.x + threadIdx.x;
7
8      if (idx < num_values) {
9          char* dst = d_output_buffer + d_output_offsets[idx];
10
11         // 第一阶段: 拷贝前缀
12         for (int i = 0; i < prefix_length; i++) {
13             dst[i] = d_prefix[i];
14         }
15
16         // 第二阶段: 拼接后缀
17         char* src = d_values[idx];
18         int value_len = d_value_lengths[idx];
19         for (int i = 0; i < value_len; i++) {
20             dst[prefix_length + i] = src[i];
21         }
22         dst[prefix_length + value_len] = '\0';
23     }
24 }
```

这个 kernel 实现了**两阶段拷贝策略**：所有线程首先并行拷贝相同的前缀（虽然是冗余读取，但前缀数据会被缓存），然后各自拼接不同的后缀。这种设计避免了复杂的同步操作，保持了高度的并行性。

3.4 线程配置

```

1  int block_size = 256;
2  int grid_size = (num_values + block_size - 1) / block_size;
3
4  generate_single_segment_kernel<<<grid_size, block_size>>>(
5      d_values, d_value_lengths, d_output_buffer, d_output_offsets, num_values);
```

选择 256 作为线程块大小是一个经过优化的配置，可以充分利用 GPU 的计算单元，实现较好的内存合并访问。同时每个线程使用的寄存器数量适中，能避免寄存器溢出。

网格大小的计算 $(\text{num_values} + \text{block_size} - 1) / \text{block_size}$ 确保了所有数据都被处理，即使数据量不是线程块大小的整数倍。

3.5 错误处理与同步

1. CUDA 错误检查

```

1  #define CUDA_CHECK(call) \
2      do { \
3          cudaError_t error = call; \
4          if (error != cudaSuccess) { \
5              std::cerr << "CUDA error at " << __FILE__ << ":" << __LINE__ \
6                  << " - " << cudaGetErrorString(error) << std::endl; \
7              exit(1); \
8          } \
9      } while(0)

```

这个宏定义为每个 CUDA API 调用提供了完备的错误检查，能够精确定位错误发生的位置，提高了代码的健壮性和可调试性。

2. 同步与数据回传

```

1  CUDA_CHECK(cudaDeviceSynchronize());
2
3  char* h_output_buffer = new char[total_output_size];
4  CUDA_CHECK(cudaMemcpy(h_output_buffer, d_output_buffer,
5                      total_output_size * sizeof(char), cudaMemcpyDeviceToHost));
6
7  output.clear();
8  output.reserve(num_values);
9  for (int i = 0; i < num_values; i++) {
10     output.emplace_back(h_output_buffer + h_offsets[i]);
11 }

```

同步机制确保 GPU 计算完全完成后再进行数据回传，避免了竞态条件。数据回传采用大块传输方式，最小化主机-设备通信开销。

4 实验结果

```
2546  Guesses generated: 9528822
2547  Guesses generated: 9699507
2548  Guesses generated: 9853408
2549  Guesses generated: 10106852
2550  Guess time:0.558367seconds
2551  Hash time:6.51058seconds
2552  Train time:29.0776seconds
2553  Cracked:358217
```

图 4.2: 实验结果

4.1 并行化效果的量化分析

由实验数据可发现，GPU 并行化带来了显著的性能改进。

对传统串行算法估算，如果使用传统 CPU 串行方法生成 1000 万密码，按照每秒处理 50 万密码的典型性能，需要约 20 秒。而 GPU 并行仅用 0.56 秒完成相同任务，实现了约 35 倍的性能提升。

同时也表明 GPU 核心利用率达到了较高的水平，内存带宽得到了有效利用，线程调度和负载均衡基本合理。

4.2 训练阶段并行性能问题

对输出日志观察可见，训练阶段存在严重的并行化问题。每个进度输出都重复 8 次，这表明系统使用了 8 个线程进行并行训练，但线程间缺乏有效的同步机制。

```
2407  Training phase 2: Ordering segment values and PTs...
2408  Lines processed: 2960000
2409  Lines processed: 2960000
2410  Lines processed: 2960000
2411  Lines processed: 2990000
2412  Lines processed: 2970000
2413  Lines processed: 2970000
2414  Lines processed: 2970000
2415  Lines processed: 3000000
2416  Lines processed: 2980000
2417  Lines processed: 2980000
2418  Lines processed: 2980000
2419  Lines processed: 3010000
2420  Training phase 2: Ordering segment values and PTs...
2421  Lines processed: 2990000
2422  Lines processed: 2990000
2423  Lines processed: 2990000
2424  Lines processed: 3000000
2425  Lines processed: 3000000
```

图 4.3: 训练阶段问题

问题表现：

- 同一进度数字重复 8 次输出，说明 8 个线程都在执行相同的进度报告
- 在高负载段（如 290000-300000 行），出现了输出混乱和重叠

根本原因分析：这种现象反映了典型的竞态条件问题。多个线程同时访问共享的输出流和计数器变量，导致输出语句的原子性被破坏，多个线程读取到相同的计数值，缺乏互斥锁保护关键共享资源。

5 实验总结

本实验成功实现了基于 CUDA 的 GPU 并行化 PCFG 密码猜测算法，将原本高度串行化的密码生成过程转换为大规模并行执行，取得了显著的性能提升。GPU 并行化策略通过扁平化内存布局、预分配输出缓冲区和批量数据传输等优化技术，实现了理想的线性加速比。密码生成阶段从原本的性能瓶颈转变为性能优势，生成速度达到千万级密码每秒的量级，相比传统 CPU 串行方法实现了数十倍的性能提升。算法在破解效果上也表现优异，验证了 PCFG 模型的有效性和 GPU 并行化策略的正确性。

同时实验过程也暴露了系统的新瓶颈和优化挑战。训练阶段的多线程并行化存在严重的竞态条件问题，从日志中大量重复输出可以看出线程间缺乏有效同步机制，导致输出混乱和资源竞争。训练阶段占据了系统执行时间的主要部分，成为新的性能瓶颈，而哈希计算也消耗了相当比例的时间。这些问题为后续优化指明了方向，需要改进训练阶段的并行化策略，解决线程同步问题，并考虑将更多计算密集型任务迁移到 GPU 执行。

6 Github 项目链接

Parallel